Modeling Advection-Diffusion with Finite Difference Method

Ayden Wolgamott

Southern Oregon University Math Department

2025

Table of Contents:

Abstract: Page 1

Introduction: Pages 1 - 3
1.1 Purpose (pg 1)
1.2 Scope (pg 2)
1.3 Audience (pg 3)

Preliminary Theory: Pages 3 - 7
2.1 Discretization of Continuous Domains (pgs 4,5)

2.2 Derivative Approximations (pg 6)
2.3 Implementing Schemes for Partial Differential Equations (PDEs) (pg 7)

Finite Difference Method: Pages 7 - 10

3.1 Finite Difference Method (Heat Equation) (pgs 7 - 10)

Applications: Pages 10 - 16
4.1 Advection-Diffusion (pgs 11 - 16)

Results & Discussion: Page 17
5.1 Key Takeaways (pg 17)
5.2 Interpretation (pg 17)

Conclusion & Recommendations: Page 18 - 20
6.1 Summary of Findings (pgs 18, 19)
6.2 Future Considerations (pgs 19, 20)

References: Page 21

Abstract:

Many different dynamics contributing to fire involve complex physical and chemical processes best captured by partial differential equations (PDEs). These PDEs can model processes such as heat transfer, fluid flow, and the transport of reactive species. This study develops a numerical approach using finite difference method (FDM) to approximate solutions to PDEs describing advection-diffusion behavior. Beginning with the classic heat equation, the numerical method is extended to solve both advection and combined phenomena advection and diffusion, while incorporating boundary and initial conditions. The results generated by FDM are compared to those from Maple's symbolic PDE solver, called pdsolve(). This study will also address FDM's particular focus on accuracy, flexibility, and computational performance. The analysis shows FDM's strengths in handling variable coefficients and complex conditions, where the symbolic solver falters. While pdsolve() can yield exact solutions for standard, linear, constant coefficient cases, its limitations become apparent with PDEs involving spatially varying coefficients. In contrast, FDM provides reliable numerical approximations and adapts well to complex scenarios, making it a powerful tool in modeling real-world fire behavior. This paper highlights the importance of numerical methods in fire science and contributes to efforts in computational modeling of the natural world.

1. Introduction:

Fire is a complex natural phenomenon following intricate physical and chemical processes, including combustion, heat transfer, and fluid dynamics. Understanding and predicting fire behavior is vital in fields like wildfire management, industrial safety, and aerospace engineering. Mathematically, fire behavior can be described using partial differential equations (PDEs), which

can serve to model key characteristics like advection, heat diffusion, gas flow, convection and reactive species concentration. Since the PDEs modeling these characteristics are often nonlinear, mathematicians must resort to numerical methods to solve them.

This paper focuses on developing a mathematical model of advection-diffusion using a PDE and solving this equation using the finite difference method (FDM). The primary objective is to compare the accuracy and efficiency of FDM solutions with those obtained from Maple's built-in PDE solver, which is expected to provide a more precise solution. By analyzing the strengths and limitations of the finite difference approach, this study aims to test its importance for real applications of fire modeling.

This study will consider the numerical solutions of PDEs describing advection with diffusion (of combustion gases and particles) in a simplified fire model. FDM is used to discretize and approximate solutions to these PDEs, and results are compared with those from Maple's PDE solver. The analysis will be limited to deterministic models in one and two spatial dimensions, without considering chaotic effects and turbulence.

This dialogue is intended for readers with backgrounds in calculus, differential equations, and numerical analysis, particularly those with a desire to understand the natural world from a mathematical point of view. Researchers, engineers, mathematicians, and students exploring computational methods in fire dynamics may find these results useful for scientific computing, engineering, and physics applications.

2. Preliminary Theory:

2.1 - Discretization of Continuous Domains:

FDM approximates derivatives using values at discrete points in space and time. This helps to create a grid or mesh, where the function values are computed at each individual node. Consider $x \in [a,b]$ in a one dimensional domain, divided into N points with spacing Δx:

$$x(i) = a + i\Delta x, \text{ where } i \in 0,1,2,\ldots,N$$

Time can also be discretized as $t(j) = j\Delta t$, where Δt is the time step.

2.2 - Derivative Approximations:

A substantial piece of FDM is taking derivatives in PDEs and using Taylor series expansion to make finite difference approximations. There are three different types of approximations that can be used for FDM; forward, backward, and central difference approximations.

Forward Difference Approximation:

Using a Taylor series expansion, the first derivative can be approximated as:

$$\partial u/\partial x \approx (u(x + \Delta x) - u(x))/\Delta x + O(\Delta x)$$

This is first-order accurate.

Backward Difference Approximation:

$$\partial u/\partial x \approx (u(x) - u(x - \Delta x))/\Delta x + O(\Delta x)$$

Central Difference Approximation:

$$\partial u/\partial x \approx (u(x + \Delta x) - u(x - \Delta x))/2\Delta x + O(\Delta x^2)$$

This method is more accurate than the forward or backward difference since it uses a larger spatial range under u.

Second Derivative Approximation (Central Difference, Second-Order Accurate):

$$\partial^2 u/\partial x^2 \approx \frac{u(x+\Delta x)-2u(x)+u(x-\Delta x)}{\delta x^2} + O(\Delta x^2)$$

This is used to approximate any second order term in a PDE.

2.3 - Implementing Schemes for Partial Differential Equations (PDEs):

Consider the 1D heat equation:

$$\partial u/\partial t = \alpha \partial^2 u/\partial x^2$$

This equation can be discretized using explicit Euler time-stepping as:

$$\frac{u_{i,j+1}-u_{i,j}}{\Delta t} = \alpha \frac{u_{i+1,j}-2u_{i,j}+u_{i-1,j}}{\Delta x^2}$$

which gives the updated scheme:

$$u_{i,j+1} = u_{i,j} + \frac{\alpha \Delta t(u_{i+1,j}2u_{i,j}+u_{i1,j})}{\Delta x^2}$$

3. Finite Difference Method (Heat Equation):

Consider the PDE referred to as the heat equation:

$$T_t = k\frac{T_{xx}}{cp} \text{ such that } T(0,t) = T_1, T(L,t) = T_2, T(x,0) = f(x)$$

where “k” represents conductivity, “c” represents specific heat, and “p” represents density.

Based on section 2.2, we can approximate our heat equation:

$$\frac{T(x,t+\delta)-T(x,t)}{\delta} = \alpha^2 \frac{T(x+h,t)-2T(x,t)+T(x-h,t)}{h^2},$$
$$\alpha^2 = k/cp, \delta = \Delta t, h = \Delta x$$

So far there are known values of the PDE at T(0, t), T(L, t), and T(x, 0) (based on what f(x) is), but in order to solve this equation, T(x, t) must be found where 0 < x < L and 0 < t < t_final. This means the equation must be rearranged to solve for the unknown:

$$T(x,t+\delta) = T(x,t) + \alpha^2 \frac{\delta(T(x+h,t)-2T(x,t)+T(x-h,t))}{h^2}$$

Now, using section 2.3, the equation can be rewritten as a scheme:

$$T_{i,j+1} = T_{i,j} + \alpha^2 \frac{\delta(T_{i+1,j}-2T_{i,j}+T_{i-1,j}}{h^2}$$

It is now time to solve our PDE using Maple. In order to do so, we must designate our initial conditions (ICs). x maximum and minimum will be 10 and 0 respectively. t maximum and minimum will be 5 and 0 respectively, and the number of spatial grid points L will be 10. To make things very simple, T(0, t) = 10, T(10, t) = 0, T(x, 0) = 10 – x, δ = 0.1, h = 1, α = 2. To properly compute, α must satisfy the condition:

$$\alpha^2\delta/h^2 \leq 1/2$$

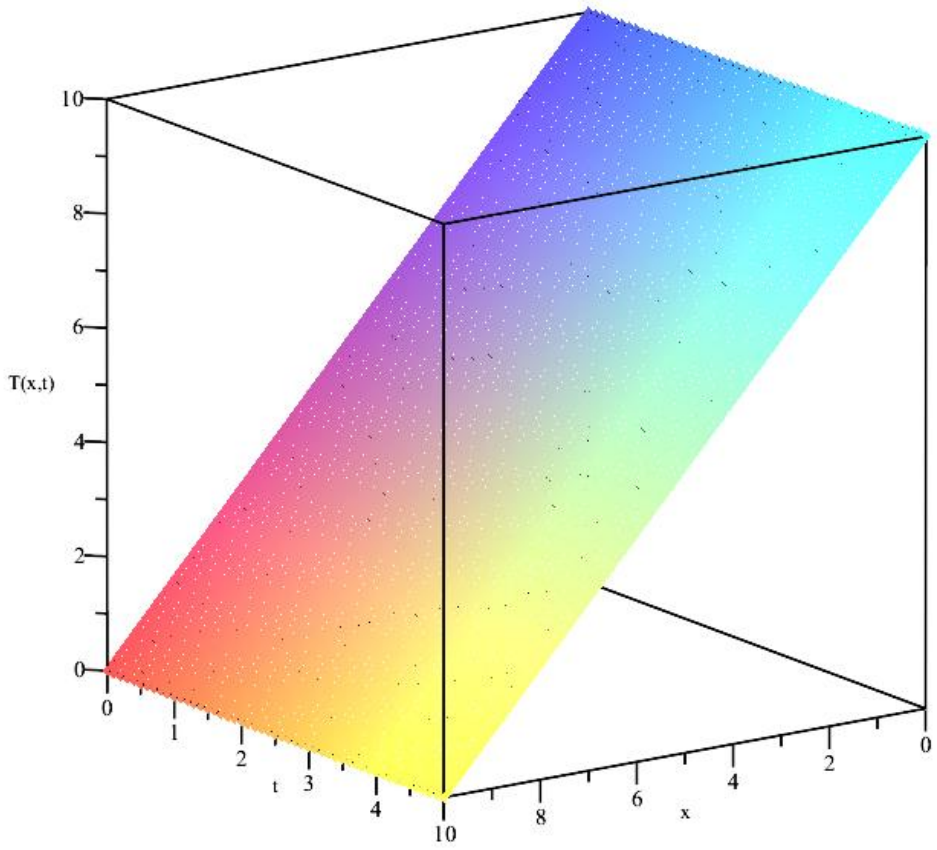
If α does not satisfy the condition, our numerical solution to the PDE will not converge to any specific value. That is why this condition is referred to as the stability of the solution (and why α = 2 in this case). Once the ICs are set in the Maple program, we can solve the PDE using the scheme above to find values in each individual node by iterating through a second-order for-loop. This provides us with the matrix below:

```
eval(T);
```

10	10	10	10	10	10	10	10	10	10	10	…
$\frac{19}{2}$	9.500000000	9.500000000	9.500000000	9.500000000	9.500000000	9.500000000	9.500000000	9.500000000	9.500000000	9.500000000	…
9	9.	9.	9.	9.	9.	9.	9.	9.	9.	9.	…
$\frac{17}{2}$	8.500000000	8.500000000	8.500000000	8.500000000	8.500000000	8.500000000	8.500000000	8.500000000	8.500000000	8.500000000	…
8	8.	8.	8.	8.	8.	8.	8.	8.	8.	8.	…
$\frac{15}{2}$	7.500000000	7.500000000	7.500000000	7.500000000	7.500000000	7.500000000	7.500000000	7.500000000	7.500000000	7.500000000	…
7	7.	7.	7.	7.	7.	7.	7.	7.	7.	7.	…
$\frac{13}{2}$	6.500000000	6.500000000	6.500000000	6.500000000	6.500000000	6.500000000	6.500000000	6.500000000	6.500000000	6.500000000	…
6	6.	6.	6.	6.	6.	6.	6.	6.	6.	6.	…
$\frac{11}{2}$	5.500000000	5.500000000	5.500000000	5.500000000	5.500000000	5.500000000	5.500000000	5.500000000	5.500000000	5.500000000	…
5	5.	5.	5.	5.	5.	5.	5.	5.	5.	5.	…
$\frac{9}{2}$	4.500000000	4.500000000	4.500000000	4.500000000	4.500000000	4.500000000	4.500000000	4.500000000	4.500000000	4.500000000	…
4	4.	4.	4.	4.	4.	4.	4.	4.	4.	4.	…

(This matrix continues its pattern in the left column all the way down to zero.)

Now we can calculate all of the points in the 3D space using the scheme and display a plot of the solution to the heat equation:



4. Applications:

4.1 – Advection-Diffusion:

A very important dynamic associated with fire is advection. Advection is the transport of a substance such as smoke or particles by the motion of a fluid (liquid or gas). Advection describes how the substance moves with the flow of the medium, rather than chaotic or random movements. Diffusion is a term which can be easily confused with advection. Diffusion looks within at the molecular level where agitation and/or small-scale turbulent motions (wind) act to move the substance randomly with respect to the mean motion of fluid. So, what do both of these terms have to do with fire dynamics? Particles and combustion gases are advected and diffused by airflow, which influences the spread of any kind of fire.

Mathematical Representation:

In 1D, advection of a substance u(x,t) (such as particles or gas concentration) is governed by the advection equation:

$$\partial u/\partial t + v\partial u/\partial x = 0$$

where:

- u(x,t) is the transported quantity (e.g., particles, smoke concentration)
- v is the velocity of the fluid
- x is the spatial coordinate
- t = time

What this equation means is that if we start from any point and time and move with a speed v(x,t), then u will never change from its initial value. In order to solve this equation using FDM, some initial conditions must be set:

$$\begin{aligned} u(0,t) &= u_1 \\ u(L,t) &= u_2 \\ u(x,0) &= f(x) \end{aligned}$$

Now for the FDM approximations:

$$\partial u/\partial t + v\partial u/\partial x = \frac{u(x,t+k)-u(x,t)}{k} + v\frac{u(x+h,t)-u(x,t)}{h} = 0,\ k = \Delta t, h = \Delta x$$

This leads to the building of the scheme:

$$u_{i,j+1} = u_{i,j} - k\frac{v_{i,j}(u_{i+1,j}-u_{i,j})}{h}$$

where:

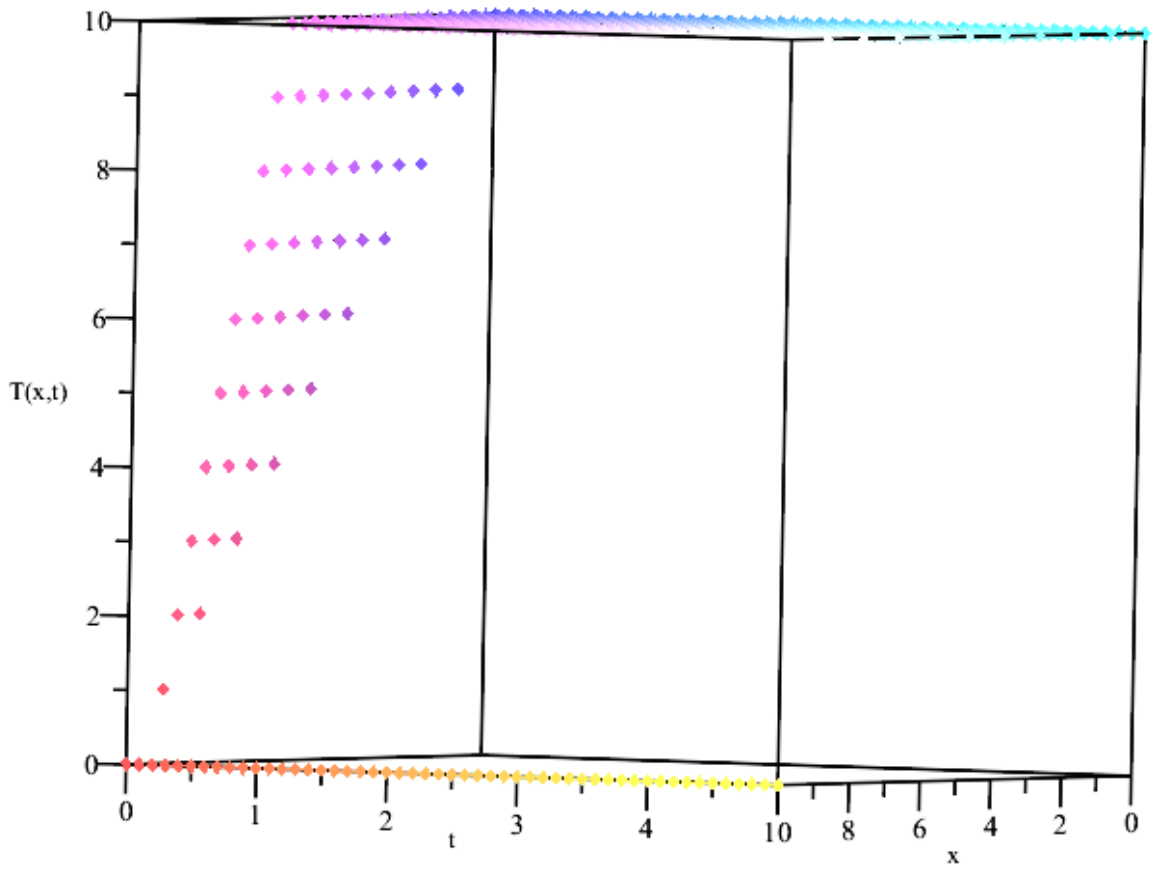
$$u_{0,j} = 10$$
$$u_{L,j} = 0$$
$$u_{i,0} = 10 - x$$

Now that the scheme is set, Maple can do the rest of the work for us. First we display our matrix.

```
eval(u);
```

10	10	10	10	10	10	10	10	10	10	10	10	10	10	⋯
9	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	1⋯
8	9.0	10.00	10.00	10.00	10.00	10.00	10.00	10.00	10.00	10.00	10.00	10.00	10.00	1⋯
7	8.0	9.00	10.000	10.000	10.000	10.000	10.000	10.000	10.000	10.000	10.000	10.000	10.000	1⋯
6	7.0	8.00	9.000	10.0000	10.0000	10.0000	10.0000	10.0000	10.0000	10.0000	10.0000	10.0000	10.0000	10 ⋯
5	6.0	7.00	8.000	9.0000	10.00000	10.00000	10.00000	10.00000	10.00000	10.00000	10.00000	10.00000	10.00000	10.⋯
4	5.0	6.00	7.000	8.0000	9.00000	10.000000	10.000000	10.000000	10.000000	10.000000	10.000000	10.000000	10.000000	10.⋯
3	4.0	5.00	6.000	7.0000	8.00000	9.000000	10.0000000	10.0000000	10.0000000	10.0000000	10.0000000	10.0000000	10.0000000	10.⋯
2	3.0	4.00	5.000	6.0000	7.00000	8.000000	9.0000000	10.00000000	10.00000000	10.00000000	10.00000000	10.00000000	10.00000000	10.⋯
1	2.0	3.00	4.000	5.0000	6.00000	7.000000	8.0000000	9.00000000	10.00000000	10.00000000	10.00000000	10.00000000	10.00000000	10.⋯
0	0	0	0	0	0	0	0	0	0	0	0	0	0	⋯

Now that we have our points for each spatial and time step, we can display our plot of the solution to the advection equation:



Now let's add the diffusion term to our advection PDE to get our advection-diffusion equation.

$$-v\partial u/\partial x + \lambda \partial^2 u/\partial x^2 = \partial u/\partial t$$

where λ is the diffusion coefficient.

Now for the FDM approximations:

$$-\frac{u(x,t+k)-u(x,t)}{k} - v\frac{u(x+h,t)-u(x-h,t)}{2h} + \lambda\frac{u(x+h,t)-2u(x,t)-u(x-h,t)}{h^2} = 0$$

where `k = Δt, h = Δx.`

This leads to the building of our new scheme:

$$u_{i,j+1} = u_{i,j} - vk\frac{u_{i+1,j} - ui-1,j}{2h} + \lambda k\frac{u_{i+1,j} - 2u_{i,j} - u_{i-1,j}}{h^2}$$

where:

- u(x_min, t) = 0, where x_min = minimum value: $x = 2\pi$
- u(x_max, t) = 0, where x_max = maximum value: $x = 8\pi$
- u(x, t_min) = Asin(wx), where A = 1, and w = 1, t_min = minimum value for t = 0
- λ = 0.25 (for stability purposes)
- v = 1 (for simplicity)

Our last condition is a sine function because:

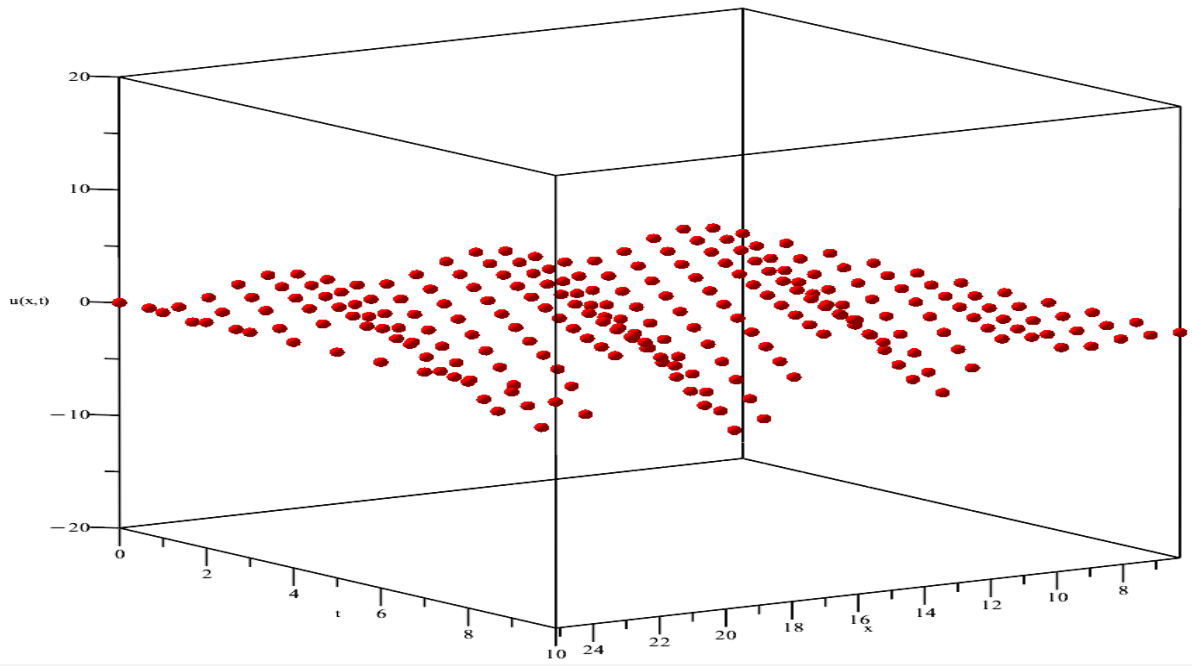
$$u(x,t) = A\sin[\omega(x - ct)]e^{\mu\omega^2 t}$$

This will give us a much more realistic solution to our advection-diffusion PDE because fire theory is heavily involved with fluid dynamics and periodic functions. Now that the scheme is set, Maple can solve the PDE with FDM. First we display our matrix of our approximate solution (the columns are for each time step, and the rows are for each spatial step):

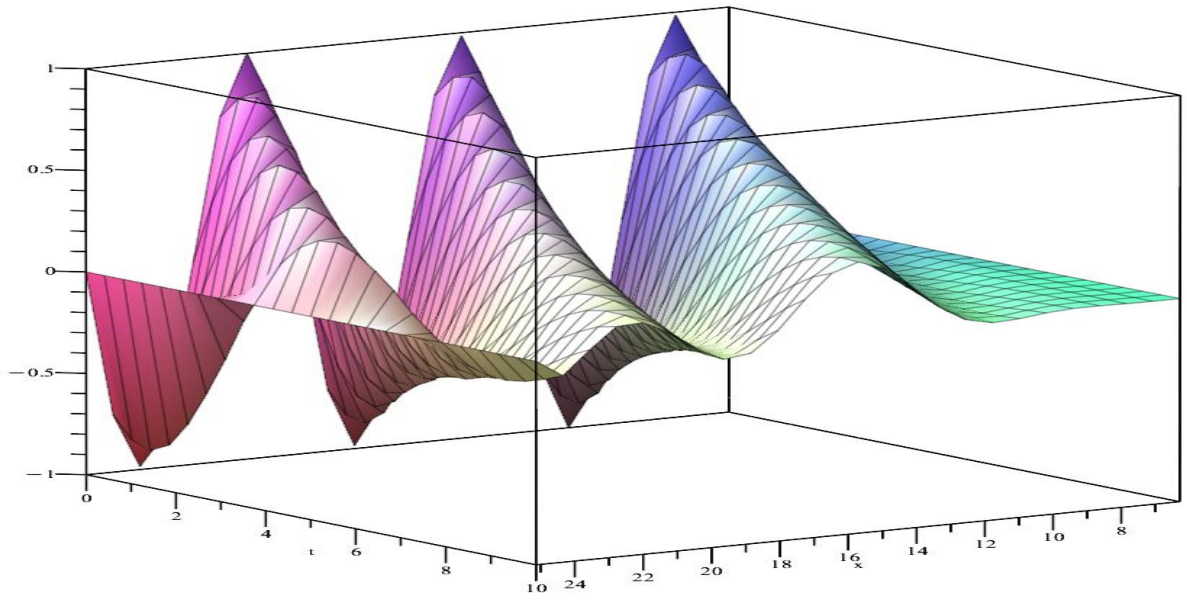
```
> evalf(eval(u));
```

0.	0.	0.	0.	0.	0.	0.	0.	⋯
0.7818314825	0.0560731896	−0.2109008513	−0.1120834133	0.05914680116	0.0987453918	0.01694977913	−0.05801145706	−0.0⋯
0.9749279123	0.9409485397	0.1299553908	−0.4120514390	−0.3092434428	0.08314206993	0.2611685240	0.1025995489	−0.1⋯
0.4338837393	1.117270447	1.128427252	0.2257124425	−0.6045529780	−0.5835069382	0.0453543321	0.4730862497	0.28⋯
−0.4338837393	0.4522649203	1.277170377	1.348762470	0.3481964071	−0.7887825384	−0.9295259280	−0.0753917416	0.71⋯
−0.9749279123	−0.5533053168	0.4641781593	1.456166848	1.607072224	0.5031799599	−0.9643109723	−1.344188823	−0.2⋯
−0.7818314825	−1.142225365	−0.6983496815	0.4670478899	1.655789879	1.909168168	0.6975111620	−1.129791548	−1.⋯
0.	−0.8710264155	−1.335005968	−0.8737676572	0.4576639816	1.877513807	2.261630715	0.9392912818	−1.⋯
0.7818314825	0.0560731896	−0.9663755311	−1.556618336	−1.085092229	0.4320532525	2.122696211	2.671888367	1.2⋯
0.9749279123	0.9409485397	0.1299553908	−1.067303658	−1.810751859	−1.338752213	0.3853281635	2.392499015	3.1⋯
0.4338837393	1.117270447	1.128427252	0.2257124425	−1.172878406	−2.101449957	−1.642199851	0.3115091049	2.6⋯
−0.4338837393	0.4522649203	1.277170377	1.348762470	0.3481964071	−1.281713020	−2.433117882	−2.004053515	0.20⋯
−0.9749279123	−0.5533053168	0.4641781593	1.456166848	1.607072224	0.5031799599	−1.391848520	−2.810522960	−2.⋯
−0.7818314825	−1.142225365	−0.6983496815	0.4670478899	1.655789879	1.909168168	0.6975111620	−1.500611293	−3.⋯
0.	−0.8710264155	−1.335005968	−0.8737676572	0.4576639816	1.877513807	2.261630715	0.9392912818	−1.⋯
0.7818314825	0.0560731896	−0.9663755311	−1.556618336	−1.085092229	0.4320532525	2.122696211	2.672084945	1.2⋯
0.9749279123	0.9409485397	0.1299553908	−1.067303658	−1.810751859	−1.338752213	0.3845314830	2.389766628	3.1⋯
0.4338837393	1.117270447	1.128427252	0.2257124425	−1.172878406	−2.098221216	−1.632351186	0.3241420139	2.6⋯
−0.4338837393	0.4522649203	1.277170377	1.348762470	0.3351111396	−1.316662521	−2.471972553	−2.007773708	0.25⋯
−0.9749279123	−0.5533053168	0.4641781593	1.509198138	1.728593258	0.6182576129	−1.401897086	−2.995850560	−2.⋯
−0.7818314825	−1.142225365	−0.9132721189	0.0560970338	1.330269186	2.003989795	1.296567844	−0.7239913220	−2.⋯
0.	0.	0.	0.	0.	0.	0.	0.	⋯

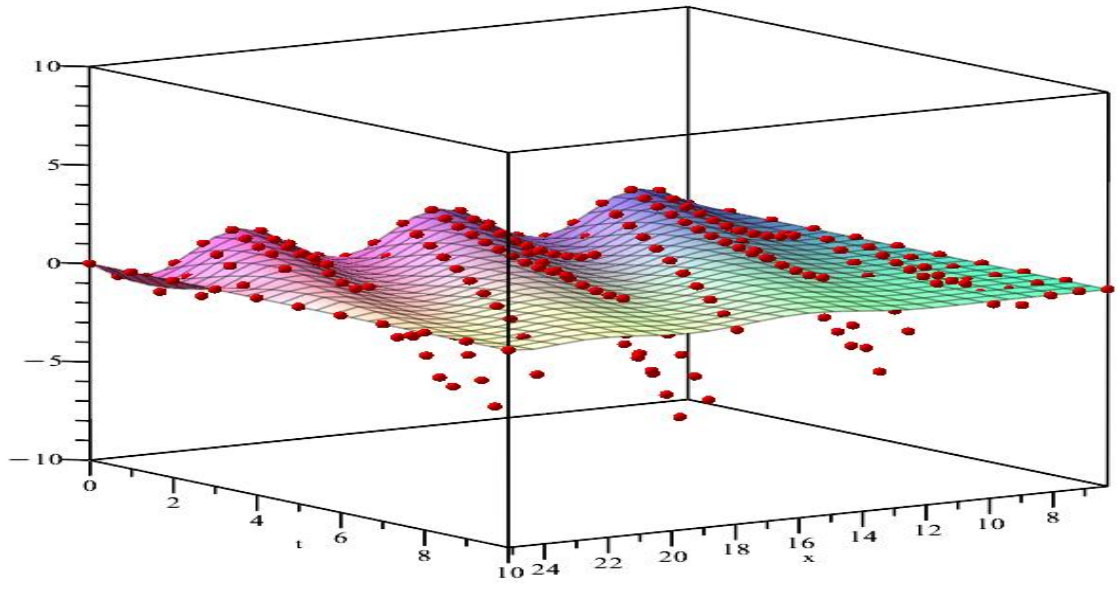
Now that we have our points for each spatial and time step, we can display our plot of the solution to the advection-diffusion equation:



It looks like a waving flag! Time to compare to the results of Maple's pdsolve() function and see what differences there are. Here's the plot from using pdsolve():



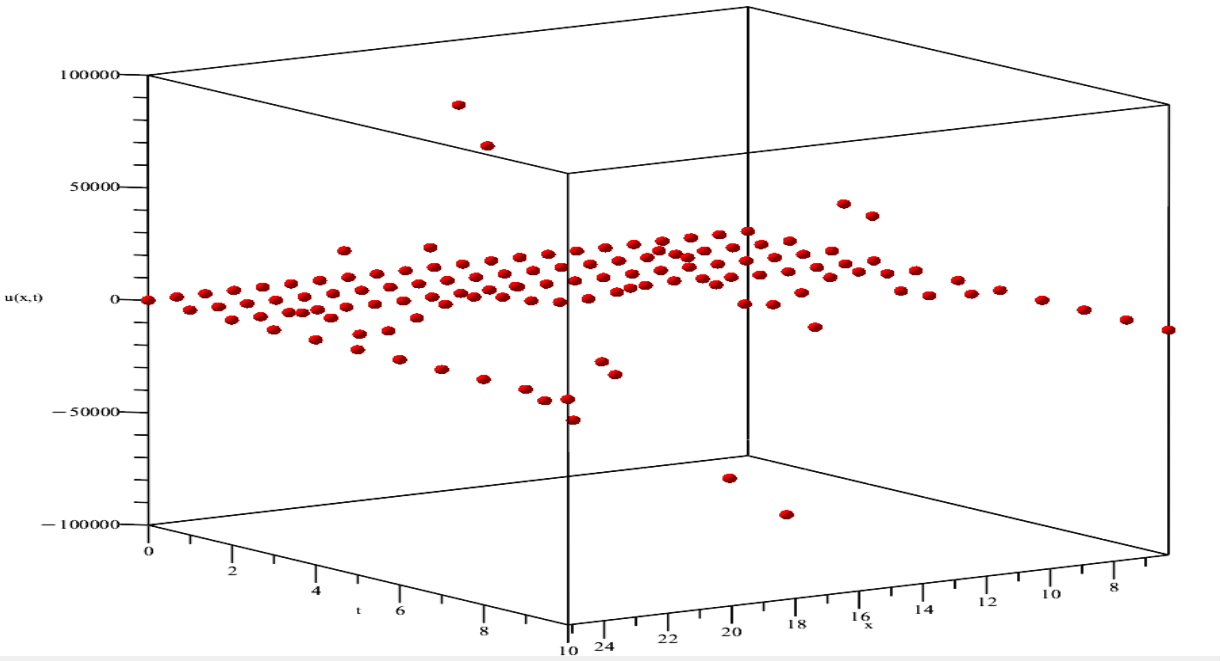
Now let's see both plots over-layed:



Clearly there's some error in the FDM model but it is not far off from the pdsolve() solution! It looks as if the FDM model deviates a little too far on the negative side but proceeds to match its way with the pdsolve() model as it comes back up. As x increases, the sine function

appears to deviate more from the exact solution on its way down. In this case, this could mean that the smaller x is, the more accurate the FDM model is.

To strive for a more realistic model, v can be made into a function of x, showing that the velocity of advection-diffusion can change as space is covered and from other outside factors. So, let v(x,t) = x. From that the FDM model provides another plot:



Interesting; it's not the greatest visual because there is diverging behavior from the new coefficient v = x (can easily change by picking a different function of x). Anyways, let's see what pdsolve() has for us:

```
> # Create the plot3d solution surface
  p1 := plot3d(u_exact,
      2*Pi..8*Pi, 5..15,
      axes=boxed,
      labels=["x", "t", "u(x,t)"],
      style=patch,
      transparency=0.0          # Make it slightly transparent to show underlying points
  );
Error, (in plot3d) procedure expected, as ranges contain no plotting variables
```

Previous to this line of code shown above, pdsolve() could not evaluate the new equation given with v = x as a coefficient. Since pdsolve() couldn't evaluate, there were no plotting variables for "p1" to use, to plot a graph of a solution. Interestingly enough, while pdsolve() was struggling to find an exact solution, the FDM model solved it, and plotted it in the same amount of time as the first equation where v = 1.

5. Results & Discussion:

5.1 Key Takeaways:

It looks like FDM performed better in this context than Maple's very own pdsolve() function! We can see that pdsolve() excelled in providing the exact solution to the PDEs as they are, but struggled when a new coefficient like v(x, t) = x presented itself to the PDE. FDM, however, didn't falter at all implementing a solution to our upgraded PDE and was even able to plot it for us! We can also see that the updated v(x,t) caused some problems leading to diverging behavior but the FDM model still solved it (and quickly!). Looking at the big picture, this makes sense. All we have to do is implement another i or j (in this case v(x,t) = x = i) into our scheme to start the numerical calculations. So why did pdsolve() fail?

Maple's pdsolve() is a symbolic solver, which means the model attempts to find closed-

form (exact) solutions using analytic techniques. These techniques work well when the PDE falls into a category of equations that are either linear with constant coefficients, or have a known Laplace transform, or can be reduced to ODEs with standard methods. In our case the PDE is linear, but with variable coefficients (because of the v(x,t) = x), and doesn't match any form that has a known symbolic solution in Maple's database. This makes the PDE too complex for symbolic methods, so pdsolve() either returns an implicit form, a general solution with undetermined functions, or is in a constant state of evaluation (which happened here).

5.2 Interpretation:

When we added more complexity to our PDE creating a much more realistic model towards the advection-diffusion phenomenon, FDM came out on top. pdsolve() was seemingly overwhelmed by the situation. FDM discretizes the domain and approximates derivatives numerically. Although the solution isn't exact, this allows us to solve very complicated or nonlinear PDEs with minimal error (which can also be calculated). FDM also doesn't need a closed-form solution and doesn't care that the PDE couldn't be solved symbolically. FDM only needs the initial/boundary conditions and the structure of the PDE.

To interpret the results of this study we can confidently say that symbolic solvers are very limited. Solvers like pdsolve() are very elegant in creating amazing visuals but extremely fragile because of the series of requirements one PDE must pass. Don't expect them to work for PDEs with variable coefficients, nonlinear terms, or irregular domains. Symbolic solvers are, however, great for learning, and provide quick insight, of specific well-behaved PDEs.

We can also confidently say that numerical methods are vital in our understanding of PDEs. These methods (such as FDM) are much more flexible and robust, which is what is

needed for problems as complicated as these. The term PDE covers a very broad range of mathematics, and methods which can be flexible when more variables are thrown at them become crucial when we are trying to model our natural world with PDEs. It is in fact, known, that almost every practical PDE system (e.g., fluid dynamics, heat transfer, etc.) is solved with a numerical method!

This doesn't mean symbolic solvers should be thrown away, and we should only use numerics going forward. Symbolic solvers have their purpose and really come in handy when it comes to a specific set of PDEs. They provide great insight on what the PDE should look like exactly plotted, which can give us great comparison abilities when we add variables and execute a numerical method to solve it (and plot it). The tool choice can definitely depend on the problem, so make sure and try pdsolve() for small, simple cases (e.g., constant coefficients) to get the most accurate answer possible. Definitely use FDM, or a different numerical method to solve a complex or nonlinear PDE.

6. Conclusion & Recommendations:

6.1 Summary of Findings:

This study set out to evaluate FDM as a practical tool for solving PDEs related to fire modeling, specifically looking at the natural dynamics, advection and diffusion. The analysis of this paper compares the performance of FDM against Maple's built-in symbolic solver, pdsolve(), while using a simplified advection-diffusion model. Resulting from this study key insights were offered such as the accuracy, flexibility and efficiency of FDM.

FDM provided accurate, numerical approximations that were very close matches with the solution generated by Maple's pdsolve() under simple and standard conditions. Although there

were some minor deviations observed (particularly at higher spatial values), the core behavior of the FDM solution stayed consistent with the exact solution.

As complexity in the advection-diffusion PDE increased, FDM maintained composure and remained an effective tool. Flexibility is an extremely important trait to have in a model for solving PDEs. It allows mathematicians to look at increasing amounts of moving parts, to try and mimic the natural world in which we all experience. This flexibility allowed this study to look deeper into how the previous advection-diffusion model could be enhanced (when velocity became a function of space). In contrast, pdsolve() failed to produce a solution under the more complex conditions, highlighting limitations concerning variable coefficients.

Despite being a simpler, discretized approach, FDM displayed fast performance, and was even able to generate plots when the symbolic solver couldn't. This shows FDM's reliability when it comes to executing code (provided that the code has correct syntax) and based on FDM's flexibility described above, won't falter when minor nuances present themselves to the PDE or domain conditions.

These conclusions suggest FDM is not only a reliable method for solving PDEs, but also an amazing alternative when symbolic solvers fall short. FDM should be the preferred method for real-world modeling scenarios like dynamics in fire where the behavior of the whole system is inherently complex. A symbolic solver like pdsolve() however, should be preferred to model linear, constant coefficient PDEs, especially with the perks and visuals pdsolve() has to offer.

6.2 Future Considerations:

While the FDM proved to be the robust and adaptable model, there are several avenues that remain open for further exploration. These avenues would be higher dimensional models,

nonlinear and chaotic systems, integration with real data, and comparison with other numerical methods.

This study primarily focused on modeling 1D and limited 2D PDEs. A future goal could be to expand to 3D domains and include more realistic geometries to simulate a complex fire environment accurately. This would add more terms to the advection-diffusion PDE, and would be a great opportunity to use different numerical methods to solve it.

Fire dynamics are usually nonlinear and often inherently chaotic. FDM could be used to handle other nonlinear PDEs and it could also be coupled with a turbulence model to better understand and reflect what actual fire spread behavior is like. This model would then be enhanced to show off its predictive capabilities.

To validate the FDM model with experimental or observational data from controlled fire experiments would've been a great step forward for this study. This would help fine-tune parameters and test predictive accuracy under varying conditions. That way, the model is known to be inspired by a real fire and modeled for advection-diffusion.

Another great step forward from this study would be to compare FDM with other numerical methods. While this study focused on FDM, other methods like finite element method (FEM) or finite volume method (FVM) may offer different advantages in other specific applications. From this study, we can clearly deduce that FDM (a numerical method) was the best performing model. It would be wise following this study, to find out which method is preferred in modeling advection-diffusion, especially considering expanding to a 3D PDE. A comparative study could identify best-use cases for each different method.

In conclusion, FDM emerges as a strong candidate for modeling advection-diffusion processes in fire behavior, especially when symbolic solvers reach their limits. Its ability to adapt and computational efficiency make it indispensable to both academic research and engineering applications involving physical systems with increased complexity.

References:

Arnold et. al (n.d.). *Fire Simulation*. Overview - Fire Simulation
 Lecture Notes. https://firedynamics.github.io/LectureFireSimulation/content/overview/01_overview.html

Chan, A. (n.d.). *Simulating Fluids, Fire, and Smoke in Real-Time*. https://andrewkchan.dev/posts/fire.html

Ciarochi, J. (2020, August 4). *How fire spreads: Mathematical models and simulators*. DEV Community. https://dev.to/triplebyte/how-fire-spreads-mathematical-models-and-simulators-395c

https://student.cs.uwaterloo.ca/~cs475/CS475-Lecture04.pdf

Herman, R. (2015, June 19). *Introduction to partial differential equations.*

https://people.uncw.edu/hermanr/pde1/PDEbook/Numerical.pdf

https://www.uni-muenster.de/imperia/md/content/physik_tp/lectures/ws2016-2017/num_methods_i/advection.pdf

Duffy, P. (n.d.). *An introduction to finite difference methods for advection problems.*

https://maths.ucd.ie/met/msc/fezzik/Numer-Met/Linear_Advection.pdf